



Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования «Уральский федеральный университет имени первого Президента России Б.Н. Ельцина» (УрФУ)
Институт фундаментального образования
Кафедра «школа бакалавриата (школа)»

ОТЧЕТ

По итоговому проекту дисциплина

Технологии распределенного реестра

по теме: **Хранилище хешей файлов для подтверждения факта
существования документа**

Студенты:

Митин Никита Андреевич

Разработчик смарт-контракта

Рассомахин Никита Андреевич

Разработчик фронтенда

(ФИО)

(Роль)

Руководитель: Саиф М.А.

Екатеринбург

2026

СОДЕРЖАНИЕ

1. Введение и список участников команды
2. Постановка задачи и обоснование выбора решения
3. Проектирование и реализация DApp-приложения
4. Тестирование смарт-контракта и интерфейса
5. Результаты работы и пример использования приложения
6. Заключение

1. ВВЕДЕНИЕ И СПИСОК УЧАСТНИКОВ КОМАНДЫ

Целью данного проекта является закрепление на практике знаний по разработке смарт-контрактов на языке Solidity, их тестированию, интеграции с веб-интерфейсом и организации полного цикла создания децентрализованного приложения (DApp). В рамках проекта реализовано хранилище хешей файлов, позволяющее подтвердить факт существования документа в определённый момент времени без раскрытия его содержимого. Приложение построено на основе стека Scaffold-ETH-2 и взаимодействует с локальной сетью Hardhat через браузерный кошелёк MetaMask.

Проект выполнен командой из двух человек с равным распределением обязанностей. Роль **разработчика смарт-контракта** выполнял Митин Никита Андреевич. В его задачи входили: проектирование и написание логики смарт-контракта на языке Solidity, реализация функций чтения и записи, создание событий и проверок безопасности, написание unit-тестов, а также подготовка скрипта деплоя в локальную сеть Hardhat.

Роль разработчика фронтенда выполнял Рассомахин Никита Андреевич. Им разработан собственный пользовательский интерфейс HashStorage на Next.js и React: загрузка файла и локальное вычисление кеша, отправка хеша в смарт-контракт через MetaMask, проверка уже сохранённого хеша и отображение истории операций. Также реализованы валидация ввода, уведомления о статусе транзакции, адаптивная вёрстка и цветовое оформление в красно-чёрно-белой гамме. Шаблонная вкладка Debug Contracts удалена.

2. ПОСТАНОВКА ЗАДАЧИ И ОБОСНОВАНИЕ ВЫБОРА РЕШЕНИЯ

Задача проекта — создать децентрализованное приложение, позволяющее любому пользователю сохранить в блокчейне хеш (кеccak256) произвольного файла или документа, а также в любой момент проверить, был ли такой хеш зафиксирован ранее. Это востребовано для подтверждения авторства, доказательства существования документа до определённой даты без публикации самого содержимого.

Выбранная тема относится к категории «Данные и контент» (п. 5 задания) и является одной из наиболее простых в реализации, что позволило сосредоточиться на качественном выполнении всех технических требований. Смарт-контракт написан на Solidity версии 0.8.20, деплой и тестирование осуществлены с помощью Hardhat, фронтенд построен на Scaffold-ETH-2 (Next.js + React). Для взаимодействия с сетью используется MetaMask.

3. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ DAPP-ПРИЛОЖЕНИЯ

3.1. Смарт-контракт HashStorage

Контракт реализует минимальный, но полноценный функционал:

- Структура данных: `mapping(bytes32 => Record)`, где `Record` содержит адрес отправителя и временную метку блока.
- Write-функция `storeHash(bytes32 _hash)` — принимает хеш, проверяет, что он не равен нулю и не был сохранён ранее, после чего сохраняет запись и генерирует событие `HashStored`.
- View-функции:
 - `isHashStored(bytes32 _hash)` — возвращает `true`, если хеш уже зафиксирован;
 - `getHashDetails(bytes32 _hash)` — возвращает адрес сохранившего и время сохранения.
- Событие `event HashStored(address indexed storer, bytes32 indexed hash, uint256 timestamp)` — эмитируется при успешном вызове `storeHash`.
- Элементы безопасности:
 - `require(_hash != bytes32(0), "Hash cannot be zero")` — предотвращает сохранение нулевого хеша;
 - `require(records[_hash].timestamp == 0, "Hash already stored")` — запрещает повторную фиксацию одного и того же хеша.

```

1  pragma solidity ^0.8.20;
2
3  contract HashStorage {
4      struct Record {
5          address storer;
6          uint256 timestamp;
7      }
8
9      mapping(bytes32 => Record) private records;
10
11     event HashStored(
12         address indexed storer,
13         bytes32 indexed hash,
14         uint256 timestamp
15     );
16
17     function storeHash(bytes32 _hash) external {
18         require(_hash != bytes32(0), "Hash cannot be zero");
19         require(records[_hash].timestamp == 0, "Hash already stored");
20
21         records[_hash] = Record({
22             storer: msg.sender,
23             timestamp: block.timestamp
24         });
25
26         emit HashStored(msg.sender, _hash, block.timestamp);
27     }
28
29     function isHashStored(bytes32 _hash) external view returns (bool) {
30         return records[_hash].timestamp != 0;
31     }
32
33     function getHashDetails(bytes32 _hash)
34         external
35         view
36         returns (address storer, uint256 timestamp)
37     {
38         Record memory record = records[_hash];
39         return (record.storer, record.timestamp);
40     }
41 }

```

Рисунок 1 — Код смарт-контракта HashStorage

3.2. Скрипт деплоя и интеграция с Hardhat

Деплой выполняется скриптом `deploy/00_deploy_HashStorage.js` с использованием `hardhat-deploy`. После деплоя автоматически обновляется файл конфигурации фронтенда `deployedContracts.ts`, что обеспечивает связь интерфейса с актуальным адресом контракта.

```
1  module.exports = async ({ deployments, getNamedAccounts }) => {  
2    const { deploy } = deployments;  
3    const { deployer } = await getNamedAccounts();  
4    await deploy("HashStorage", {  
5      from: deployer,  
6      log: true,  
7    });  
8  };  
9  
10 module.exports.tags = ["HashStorage"];
```

Рисунок 2 — Скрипт деплоя

3.3. Разработка пользовательского интерфейса

Фронтенд не использует штатную отладочную вкладку Scaffold-ETH-2. Для проекта создана отдельная прикладная страница `HashStorage`. На ней реализованы:

- область выбора файла и расчёт кешак256-хеша в браузере без передачи содержимого файла в сеть;
- кнопка копирования хеша и отправка `storeHash` через `MetaMask`;
- форма ручной проверки `bytes32`-хеша с выводом адреса отправителя и времени фиксации;
- список последних событий `HashStored`;
- состояния загрузки, успеха и ошибки, а также адаптивное отображение на мобильных устройствах.

4. ТЕСТИРОВАНИЕ СМАРТ-КОНТРАКТА И ИНТЕРФЕЙСА

4.1. Unit-тесты контракта

Разработаны и выполнены 7 тестов в файле `test/HashStorage.test.js`:

1. Успешное сохранение валидного хеша и проверка эмиссии события `HashStored`.
2. Проверка `require` при сохранении нулевого хеша — ожидается откат с сообщением «Hash cannot be zero».
3. Проверка `require` при повторном сохранении того же хеша — ожидается откат с сообщением «Hash already stored».
4. Проверка, что для несохранённого хеша `isHashStored` возвращает `false`.
5. Проверка, что для сохранённого хеша `isHashStored` возвращает `true`.
6. Проверка корректности возвращаемых значений `getHashDetails` (адрес отправителя и временная метка больше нуля).
7. Проверка эмиссии события с точным соответствием параметров.


```

1  const { expect } = require("chai");
2  const { ethers } = require("hardhat");
3
4  describe("HashStorage", function () {
5    let HashStorage, hashStorage, owner, addr1;
6
7    before(async () => {
8      [owner, addr1] = await ethers.getSigners();
9      HashStorage = await ethers.getContractFactory("HashStorage");
10     hashStorage = await HashStorage.deploy();
11     await hashStorage.waitForDeployment();
12   });
13
14   describe("storeHash", function () {
15     it("should store a valid hash and emit HashStored", async () => {
16       const hash = ethers.keccak256(ethers.toUtf8Bytes("test"));
17       await expect(hashStorage.storeHash(hash))
18         .to.emit(hashStorage, "HashStored")
19         .withArgs(owner.address, hash, (timestamp) => timestamp > 0);
20     });
21
22     it("should revert when storing zero hash", async () => {
23       await expect(
24         hashStorage.storeHash(ethers.ZeroHash)
25       ).to.be.revertedWith("Hash cannot be zero");
26     });
27
28     it("should revert when storing the same hash twice", async () => {
29       const hash = ethers.keccak256(ethers.toUtf8Bytes("unique"));
30       await hashStorage.storeHash(hash);
31       await expect(
32         hashStorage.storeHash(hash)
33       ).to.be.revertedWith("Hash already stored");
34     });
35   });
36
37   describe("isHashStored", function () {
38     it("should return false for non-stored hash", async () => {
39       const hash = ethers.keccak256(ethers.toUtf8Bytes("unknown"));
40       expect(await hashStorage.isHashStored(hash)).to.equal(false);
41     });
42
43     it("should return true for stored hash", async () => {
44       const hash = ethers.keccak256(ethers.toUtf8Bytes("known"));
45       await hashStorage.storeHash(hash);
46       expect(await hashStorage.isHashStored(hash)).to.equal(true);
47     });
48   });
49
50   describe("getHashDetails", function () {
51     it("should return correct storer and timestamp", async () => {
52       const hash = ethers.keccak256(ethers.toUtf8Bytes("detail"));
53       await hashStorage.storeHash(hash);
54       const [storer, timestamp] = await hashStorage.getHashDetails(hash);
55       expect(storer).to.equal(owner.address);
56       expect(timestamp).to.be.gt(0);
57     });
58   });
59
60   describe("event HashStored", function () {
61     it("should emit event with correct parameters", async () => {
62       const hash = ethers.keccak256(ethers.toUtf8Bytes("event_test"));
63       const tx = await hashStorage.storeHash(hash);
64       await expect(tx)
65         .to.emit(hashStorage, "HashStored")
66         .withArgs(owner.address, hash, await ethers.provider.getBlock("latest").then(b => b.timestamp));
67     });
68   });
69 });

```

Рисунок 3 — Unit-тесты

Все тесты пройдены успешно. Результат прогона:

```
C:\Users\mrmit\Desktop\my-dapp>yarn test
Warning: SPDX license identifier not provided in source file. Before publishing, consider adding a comment containing
more information.
--> contracts/YourContract.sol

Compiled 1 Solidity file successfully (evm target: paris).

HashStorage
  storeHash
    ✓ should store a valid hash and emit HashStored
    ✓ should revert when storing zero hash
    ✓ should revert when storing the same hash twice
  isHashStored
    ✓ should return false for non-stored hash
    ✓ should return true for stored hash
  getHashDetails
    ✓ should return correct storer and timestamp
  event HashStored
    ✓ should emit event with correct parameters

7 passing (184ms)
```

Solidity and Network Configuration					
Solidity: 0.8.30	Optim: false	Runs: 200	viaIR: false	Block: 60,000,000 gas	
Methods					
Contracts / Methods	Min	Max	Avg	# calls	gas (avg)
HashStorage					
storeHash	-	-	68,542	7	-
Deployments					
HashStorage	-	-	347,536	0.6 %	-
Key					
⚠ Execution gas for this method does not include intrinsic gas overhead					
⚠ Cost was non-zero but below the precision setting for the currency display (see options)					
Toolchain: hardhat					

Рисунок 4 — Результаты прогона тестов

4.2. Ручное тестирование интерфейса

Проведено ручное тестирование с фиксацией ключевых этапов. Ниже приведены скриншоты с описанием действий и результатов.

Первый этап — проверка собственного интерфейса. На скриншоте должны быть видны логотип HashStorage, подключённый кошелёк, выбранный файл и вычисленный кескак256-хеш.

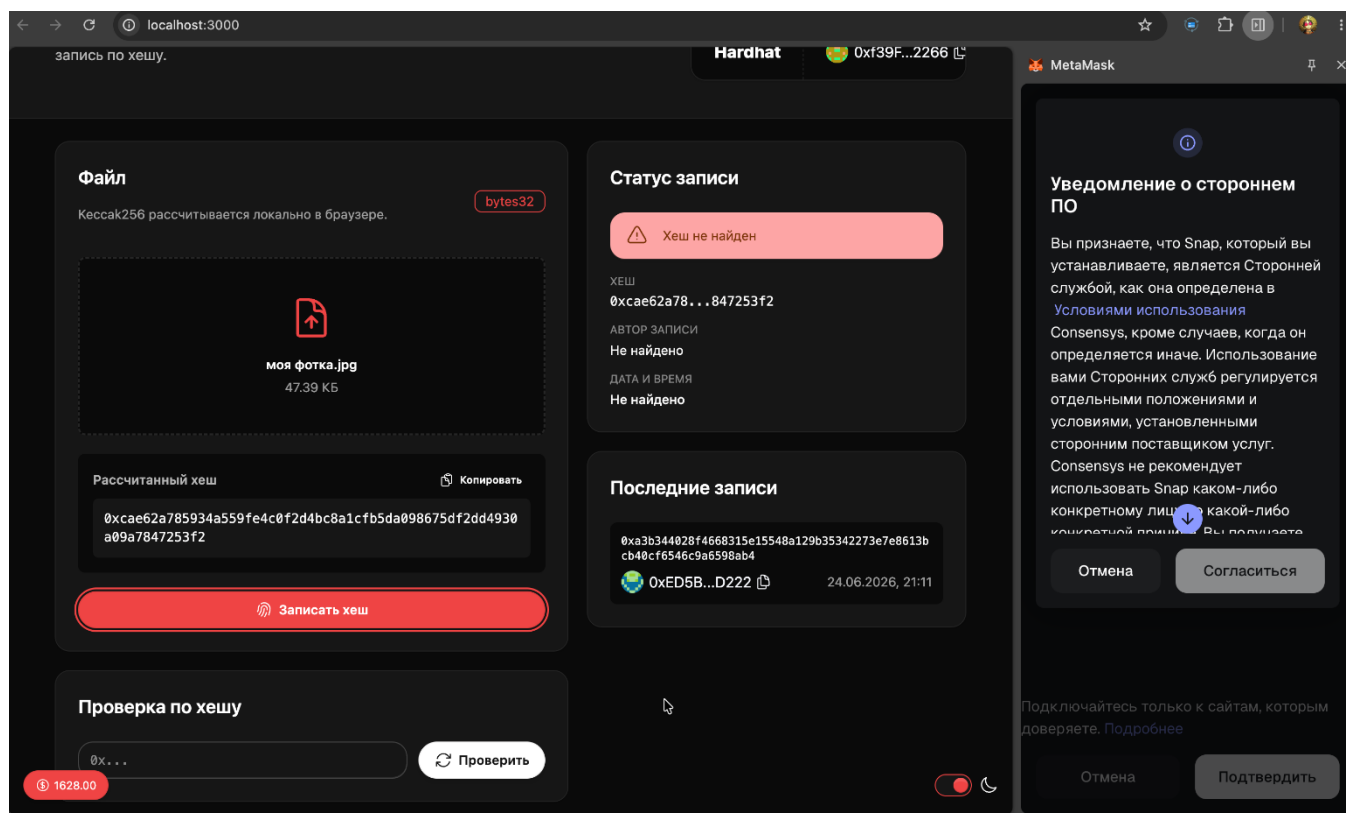


Рисунок 5 — Загрузка файла и расчёт хеша

Действие: Пользователь подключил MetaMask к локальной сети, выбрал документ в области загрузки.

Результат: Интерфейс локально рассчитал хеш файла и активировал кнопку его фиксации в блокчейне.

4.2.1. Подписание и отправка транзакции

После нажатия кнопки «Записать хеш» кошелёк MetaMask показывает вызов storeHash, адрес контракта и оценку комиссии. Пользователь подтверждает операцию.

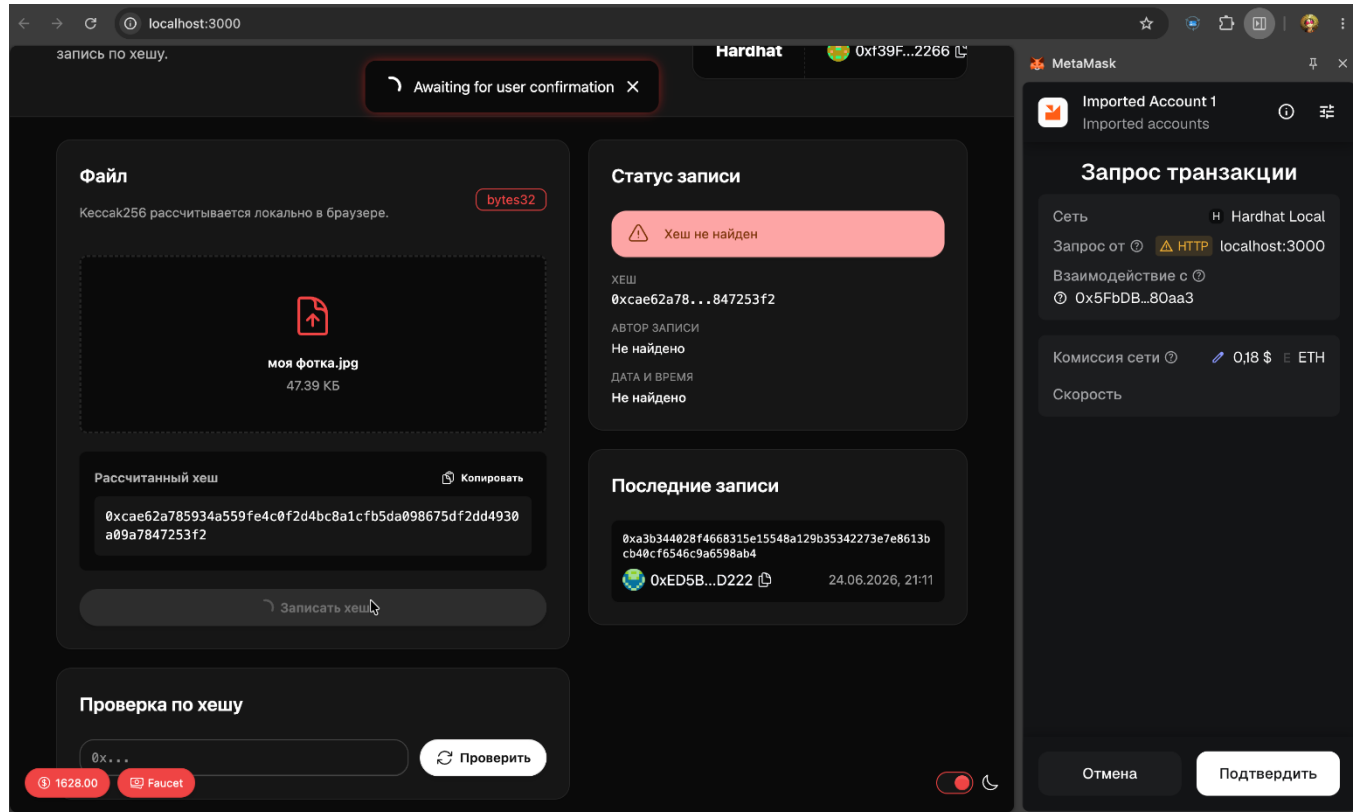


Рисунок 6 — Подтверждение транзакции в MetaMask

Действие: Открыто окно MetaMask, проверены параметры вызова смарт-контракта, нажата кнопка подтверждения.

Результат: Подписанная транзакция отправлена в локальную сеть Hardhat. Интерфейс перешёл в состояние ожидания подтверждения.

4.2.2. Успешная фиксация и проверка хеша

После включения транзакции в блок интерфейс выводит уведомление об успехе. В истории появляется событие HashStored, а повторная проверка возвращает адрес кошелька и время сохранения.

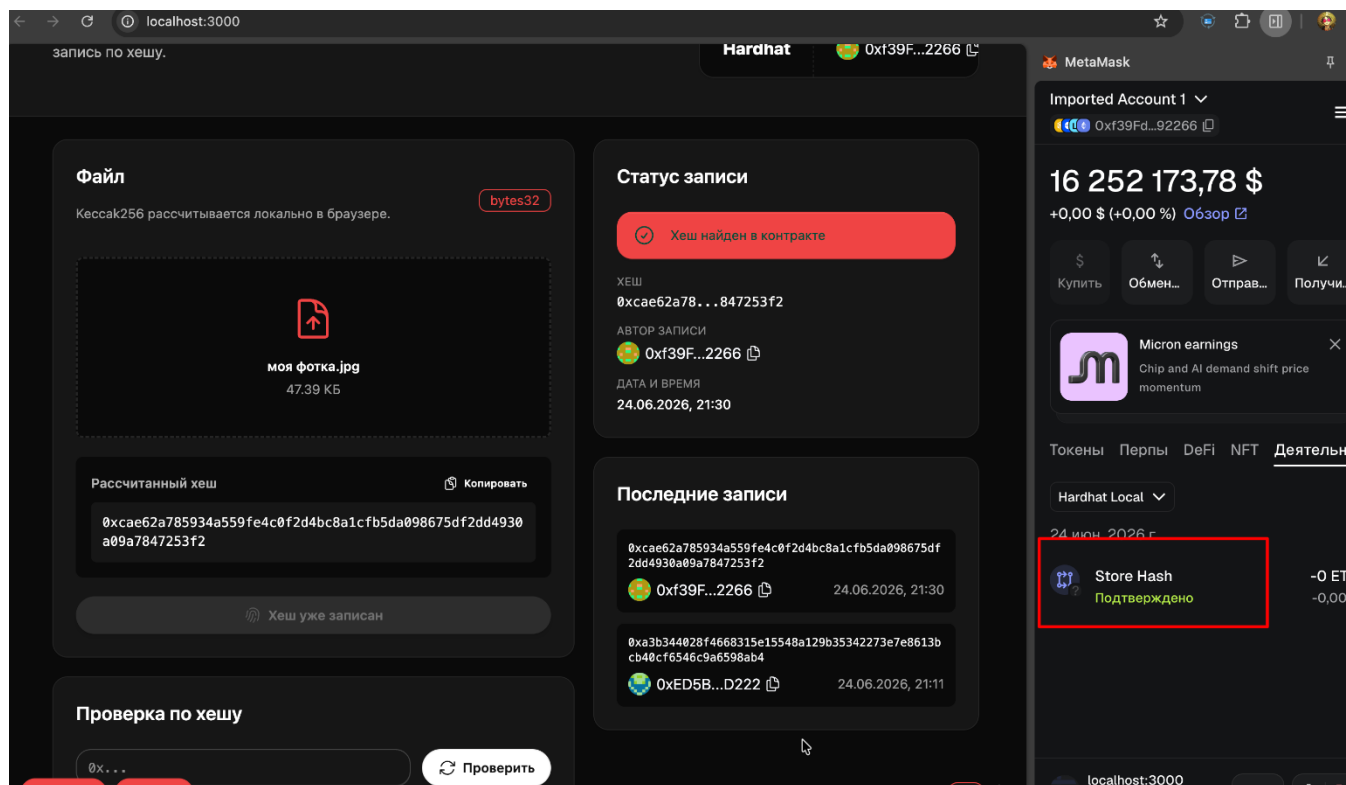


Рисунок 7 — Успешное сохранение и проверка хеша

Действие: Дождались подтверждения, затем ввели тот же bytes32-хеш в форму «Проверка по хешу».

Результат: Запись найдена. На экране отображены хеш, адрес отправителя, время фиксации и новое событие HashStored. Это подтверждает корректную работу всех фронтенд-сценариев.

5. РЕЗУЛЬТАТЫ РАБОТЫ И ПРИМЕР ИСПОЛЬЗОВАНИЯ ПРИЛОЖЕНИЯ

Разработанное децентрализованное приложение «Хранилище хешей» позволяет любому пользователю, имеющему MetaMask и доступ к сети, зафиксировать факт существования документа путём отправки хеша в смарт-контракт. В дальнейшем любое заинтересованное лицо может проверить этот факт, вызвав `isHashStored`. Приложение полностью удовлетворяет техническим требованиям:

- Реализованы две `view`-функции и одна `write`-функция.
- Генерируется событие при сохранении.
- Присутствуют проверки безопасности (`require`).
- Написаны и успешно выполнены `unit`-тесты, покрывающие `write`-функцию, событие и ошибочные сценарии.
- Разработан собственный фронтенд `HashStorage`: выбор файла, расчёт хеша, отправка транзакции, проверка записи и история событий.
- Интерфейс интегрирован с MetaMask и отображает статусы операций. Ручное тестирование описано в разделе 4.2.

Ссылка на репозиторий GitHub: <https://github.com/mrmitin78-arch/Final>

Пример использования: пользователь хочет заверить факт создания научной статьи. Он вычисляет `keccak256`-хеш файла статьи и отправляет его в контракт. Хеш и временная метка сохраняются. Позднее, при возникновении спора об авторстве, можно предъявить хеш и доказать, что документ существовал на момент транзакции.

6. ЗАКЛЮЧЕНИЕ

В ходе выполнения итогового проекта по дисциплине «Основы технологии блокчейн» были успешно применены на практике навыки разработки смарт-контрактов на Solidity, их тестирования с использованием Hardhat, создания децентрализованного приложения на базе Scaffold-ETH-2 и интеграции с кошельком MetaMask. Реализованное хранилище хешей является минимальным, но полнофункциональным DApp, отвечающим всем заявленным требованиям. Проект может быть расширен возможностью хранения дополнительных метаданных, интеграцией с IPFS или поддержкой нескольких сетей.